

Canonical Dataset Lineage for Decentralized Agent Networks

Maura Clark
Kessai Labs
Napa, CA, USA
maura@kessailabs.com

Abstract—Agents in decentralized networks re-export, derive, refresh, and hand off datasets. Identifiers computed over serialized bytes fork under these operations: the same logical dataset, re-encoded by a different tool, acquires a new identity, severing its lineage and freshness history. We present a protocol that gives equivalent encodings of a dataset one content-derived identifier, binds each derived dataset to canonical references to its parents, authenticates publication by the dataset’s owner, and takes freshness from an append-only transparency log rather than from producer clocks. The protocol combines existing IETF mechanisms for canonicalization, signed statements and transparency services. The requirements come from maintainer reviews of five contributions to NANDA Town, an open testbed for agent protocols. In a standalone prototype, the canonicalization pipeline gives all seven supported re-encoding cases the same identifier, where Town’s existing record fingerprint gives the same identifier to only three of the seven. It also rejects all 21 preregistered invalid inputs with the expected error classifications. The measurements cover canonicalization only; signed publication, lineage traversal and freshness have since been exercised in a standalone Lab prototype, and testbed integration remains to be evaluated.

Index Terms—agent networks, provenance, content addressing, canonicalization, transparency logs, SCITT

I. INTRODUCTION

Networked agents do not just exchange messages; they pass data. A procurement agent exports a parts list; a planning agent re-imports it through a different JSON library, derives an assembly plan, and publishes that as a new dataset; an auditor later asks which inputs the plan came from and whether they were current. Each step is routine. Yet if dataset identity is a hash of the bytes each tool happened to emit, every re-encoding mints a new identifier, and lineage, freshness, and any claim bound to the old identifier silently detach. An adversary can exploit the same property deliberately, re-encoding a dataset to break the link to its recorded history.

The pieces exist. RFC 8785 JCS [1] fixes key order and number serialization but performs no Unicode normalization, by design, so the composed (NFC) and decomposed (NFD) forms of the same text [6] hash differently. The IETF architecture for supply chain integrity, transparency and trust (SCITT) [2] provides signed statements and append-only Transparency Services, and the Canonical Payload Binding (CPB) draft [3] defines how a payload profile declares a canonicalization, derives an identifier, and cites other artifacts by typed digest reference. We are not aware of a *dataset* profile

that combines them to provide an identity that survives re-encoding, parent references a verifier can traverse, ownership bound by signature, and freshness backed by log evidence.

This paper contributes: (i) requirements drawn from what the maintainers of NANDA Town (“Town”) [4] wrote when they closed five contributions, rather than from first principles; (ii) a dataset profile built on SCITT and CPB that is designed to meet them, running the canonicalization steps in a fixed order and stating exactly which log evidence establishes ordering and freshness; (iii) a test design in which every test case those maintainers kept becomes a check the evaluator reports, alongside a deliberately weak control that must fail; and (iv) measurements from a prototype of the canonicalization step.

II. PROBLEM AND REQUIREMENTS

A. Identity forks in practice

Town’s evidence layer identifies records by `records.fingerprint`: SHA-256 over JSON with sorted keys and compact separators [4]. That is the right design for Town: it fingerprints the parsed record model that Town’s HMAC signatures cover. It is insufficient, however, as a persistent *dataset* identifier, because datasets are re-encoded as they move between tools. We call two encodings *profile-equivalent* when they differ only in ways the pipeline of §III-A absorbs: member order, insignificant whitespace, the spelling of a number, and Unicode normalization form. Table I shows one dataset in four profile-equivalent encodings. Sorted keys absorb reordering, but integer-to-float re-encoding and Unicode NFD each fork the identifier. JCS eliminates the numeric difference; canonically equivalent Unicode strings additionally require NFC normalization.

B. Requirements from review

We examined maintainer review of five closed 2026 contributions to Town. The maintainers closed them as groundwork for future profiles rather than as defect reports; each closing comment restated requirements and kept test vectors [5]. One of the five proposed serialization-invariant data facts, an identifier of the kind this profile specifies; the requirements below come from the closing reviews rather than from that proposal. Three of the five define the dataset contract, which we adopt as requirements: **R1** re-encodings of the same logical dataset resolve to one identity, and a single leaf mutation changes

TABLE I

ONE DATASET VALUE,
 {"UNIT": "KG", "ROWS": [[1, 2], [3, 4]], "LABEL": "Café"}, IN
 FOUR PROFILE-EQUIVALENT ENCODINGS; DIGEST PREFIXES, BOLD MARKS
 A FORK. FINGERPRINT IS TOWN'S RECORDS. FINGERPRINT AT
 DF0B5F1 (SORTED MEMBER NAMES, COMPACT SEPARATORS, UTF-8
 EMITTED DIRECTLY RATHER THAN \u-ESCAPED). THE COLUMNS ARE
 DIGEST RECIPES OVER A DATASET VALUE, NOT THE PAYLOAD PROFILE OF
 §III-A. THE INTEGRAL-VALUED-FLOAT ROW WRITES EVERY INTEGER IN
 THE VALUE AS N.0; REGENERATED BY TABLE1.PY [20].

Encoding	fingerprint	JCS	NFC→JCS
original	965293f8	965293f8	965293f8
members reordered	965293f8	965293f8	965293f8
integral-valued floats	6332808b	965293f8	965293f8
"Café" in NFD	30d12650	30d12650	965293f8

it; **R2** empty inputs receive explicit, non-colliding handling; **R3** one canonical encoding and content-derived identity, with the owner bound by signature; **R4** parents are independently traversable, supported representations of a parent reference canonicalize to the same value, conflicting ones are rejected, and a parent not bound into the canonical bytes never affects provenance; **R5** freshness is verified cryptographically, not inferred from field presence, and never depends on a writer's private clock; and **R6** a dataset type carries publication, life-cycle, verification, and evaluator semantics, with a broken-link control. The other two, on interview evidence and on outcome-gated payments, are downstream profiles that compose on this one: a quote binds to a transcript dataset, and an evaluator criterion is committed as a dataset reference.

C. Threat model

The adversary may re-encode and republish any dataset, replay old but validly signed statements, present forged or dangling parent references, and claim another party's ownership. It cannot forge an owner's signature or a Transparency Service receipt. Signatures and receipts establish commitment and registration, not truth: a producer can faithfully sign a false dataset, and we do not address that.

III. PROTOCOL

A. Canonical identity

A dataset payload has four members: `content`, `owner`, `parents`, and `schema_version`. Following CPB, the payload class declares exactly one canonicalization algorithm, CPB `jcs`, preceded by a profile-defined transformation. CPB requires the profile to fix the order of that transformation relative to the removal of excluded fields [3]; ours is:

- 1) *Admit and validate* the input bytes: strict UTF-8 without a byte-order mark, well-formed JSON, no duplicate member names after escape decoding, no non-scalar Unicode. Each number token, judged on its decimal spelling before any binary conversion, must be mathematically integral (`NUMBER_NOT_INTEGRAL`) and within $\pm(2^{53} - 1)$ (`NUMBER_OUT_OF_RANGE`).
- 2) *Transform the complete payload*: normalize every string and member name to NFC [6]; reject names that collide

after normalization; enforce a closed schema, admitting only the four members plus the legacy parent form and the excluded address, with owner a non-empty string and `schema_version` "1"; normalize parent representations and set semantics (§III-B). Validation precedes exclusion, so excluded fields cannot hide malformed input.

- 3) *Exclude* the top-level address member, which carries the identifier itself.
- 4) *Apply* RFC 8785 JCS, SHA-256, and lowercase hex.

Thus 1, 1.0, and 1e0 are one value, as R1 requires, while every non-integral quantity is carried as a string, consistent with I-JSON's advice for exact values [7]. No absent-field normalization is applied: {}, [], "", and null are distinct content (R2). A dataset without content is rejected. A record identified by Town's existing fingerprint belongs to a separate artifact type, `dataset-fingerprint`, with its own declared *digest context*: the algorithm, transformation, and representation under which its digests are computed. CPB permits a digest comparison only under compatible declared contexts, never by equal-looking hexadecimal, so identifiers of the two types are never treated as equivalent [3].

Why integral values only. Plain JCS admits any IEEE-754 binary64 value and so inherits input rounding [1]: our prototype confirmed that 0.1 and 0.10000000000000001 would share an identity. Two distinct published decimal values would then be indistinguishable under the identifier, which is the failure a lexical admission boundary prevents; the $1 \leftrightarrow 1.0$ collapse, by contrast, is the intended equivalence. Refusing every decimal-point token avoids the aliasing but also refuses 1.0; the UOR Foundation's `uor-jcs-nfc` context takes that route and renders digests with a `sha256:` prefix [11]. Taking integral values in any spelling keeps R1's $1 \leftrightarrow 1.0$ invariance without aliasing. The two contexts stay distinct and are never interchangeable.

B. Typed parent references

Each dataset carries canonical references to its parent datasets. We encode them as CPB typed digest references: closed objects carrying exactly `type: "dataset"`, `digest_alg: "SHA-256"`, and a 64-hex digest, with purpose omitted because the type has one digest context. References are carried only in the payload, never also in a `cpb-refs` header, and are not excluded, so a child's identifier commits to its parents. A legacy single-parent form maps onto the set; equivalent parent representations canonicalize to the same parent set; conflicting or duplicated ones are rejected; the set is sorted, so order never matters. Lineage is computed only from this canonical set, so a parent named anywhere else creates no edge (R4). A verifier resolves each parent by its claimed digest, fetches that dataset, and recomputes its identifier. Each edge is then reported in a CPB reference state [3]: *Verified* when the recomputed identifier matches, *Failed* when it does not, *Unresolved* when the parent cannot be fetched, *Malformed* when the reference itself is invalid. Because every identifier commits to its parents' identifiers, constructing a

valid cycle would require satisfying a circular set of SHA-256 preimage constraints, which we assume computationally infeasible; a revisited identifier therefore indicates an invalid or forged reference, and traversal reports a fifth, profile-defined state, *Cyclic*, and stops. *Malformed* and *Cyclic* are defensive: neither can arise from a payload admitted by §III-A. No state other than *Verified* counts as a pass.

C. Signed publication

Publication is an RFC 9943 Signed Statement [2] (COSE_Sign1 [8]) in full-content mode, meaning the dataset itself is the signed payload, with protected `content_type` `application/json`; the CBOR Web Token issuer claim (`iss`) is the owner and the subject claim (`sub`) the derived identifier. A declared owner that differs from the issuer is rejected (R3). Verifiers report signature validity, issuer authentication, and content binding separately, and always recompute the identifier: a supplied identifier is never trusted.

D. Receipt-backed freshness

Inclusion does not imply freshness. A verified receipt proves that a statement was registered [9], but not that no newer statement exists, and CPB content binding establishes neither [3]. The profile therefore keeps three kinds of evidence separate. *Order*: a Transparency Service keeps registered statements in one append-only Statement Sequence [2], and only a verifiable position in it establishes supersession; for example, `RFC9162_SHA256` inclusion proofs expose a leaf index and tree size, and consistency proofs to a signed current tree head establish current state [10]. *Completeness*: establishing freshness needs an authenticated view of the current log and a complete, auditable way to discover every freshness statement for the (owner, identifier) pair; the current one is then the owner-authenticated statement with the latest verified log position. Replaying an old statement cannot restore freshness, non-owners cannot refresh, and producer timestamps never order anything (R5). Elapsed-time windows additionally require registration time authenticated by the Transparency Service.

IV. TOWN CASE STUDY AND EVALUATION DESIGN

The protocol above is independent of any one platform; we use NANDA Town as the evaluation environment. Town is an open testbed for decentralized agent protocols, with twelve replaceable protocol layers, logical time with no wall clock inside a run, and validators that must judge solely from an exported event log [4]. The planned integration adds a data-facts plugin, `dataset.v1`, beside the existing evidence plugin, whose fingerprints stay unchanged. Town’s HMAC auth plugin cannot produce third-party-verifiable statements, so an asymmetric signing plugin is required, and in Town’s local simulation mode (the Lab) an append-only log stands in for a Transparency Service.

The pilot reuses Town’s supply-chain scenario: suppliers publish part datasets, a manufacturer publishes an assembly whose parents are those parts, and an auditor re-encodes, mutates, traverses, replays, and checks freshness.

TABLE II
STANDALONE PROTOTYPE OF THE CANONICALIZATION PIPELINE VERSUS TOWN’S FINGERPRINT (CONTROL).

Check	NFC→JCS	fingerprint
Re-encodings, one identity	7/7	3/7
Parent forms, one identity	3/3	—
Mutations, identity moved	4/4	—
Empty values, distinct ids	4/4	—
Invalid inputs rejected	21/21	—

A validator reports eight stages: input validation, canonical identity, mutation sensitivity, empty input, signed publication, lineage, freshness, and controls. The control plugin, `dataset-fingerprint.v1`, identifies datasets by Town’s existing fingerprint and must fail canonical identity; if it passes, the scenario is not exercising re-encoding. All sixteen review vectors retained by the three contract-defining contributions map to a stage; for example, “stale replay” is a freshness fault and “phantom parent” a lineage fault.

What is implemented and what is specified. Two standalone prototypes implement §III-A–III-D and are exercised against preregistered vectors (Table II; [20]). The Town plugin, its validator and the in-Town control are specified here and not yet built; no measurement is reported for them. We state the boundary explicitly rather than let the protocol’s scope stand in for the prototypes’.

Canonicalization prototype results (not the Town plugin). The first prototype implements the four-step canonicalization pipeline, including shape checks and parent normalization; we ran it against Town’s fingerprint as the control (Table II). The pipeline produced no forks across seven re-encoding cases (key order, whitespace, `1.0`, `1e0`, NFD in a value, NFD in a member name, `\u` escapes); the control forked on four. Reordered parents, a legacy single parent, and two representations denoting the same parent each resolved to one identity; a carried `address` was excluded and a wrong one detected; every mutation moved the identifier; the four empty values stayed distinct; and all 21 preregistered invalid inputs were rejected with the expected error classifications, nineteen of them distinct. The expected outcomes in Table II were generated by the same implementation, so they show that the tested refusal paths are reachable and return their expected error classifications, not that the implementation is independently correct; the comparative evidence is the control column, where Town’s fingerprint gives one identifier to three of the seven re-encodings. The 47 vectors are provided as exact input bytes with expected outcomes, alongside the prototype and a runtime receipt [20]. A companion paper [16] applies this canonicalization to a deployed agent data layer and reports its adoption cost and an agent-runtime study; this paper contributes the profile, the requirements derived from maintainer review, and the prototypes.

A standalone Lab prototype, not the Town plugin, matches all 51 pinned vectors [20]; its owner binding, idempotent log and nonce are Lab-model choices, not protocol rules.

V. RELATED WORK

SCITT [2] and Certificate Transparency [10] supply signed statements and append-only logs; CPB [3] supplies the canonicalize-derive-cite construction on which our profile is built. A companion CPB payload profile, the Agent Action Capsule [15], gives an agent’s *actions* a signer-independent content identity under the same construction: a Capsule chains to a single prior Capsule and cites external artifacts by typed digest reference. We identify the *datasets* those actions read and write, bind the owner into the identity, and commit a set of parents rather than one. Software supply-chain systems such as in-toto [12] and Sigstore [13] bind attestations to artifact digests but identify artifacts by bytes, which suits build outputs and fails for structured data re-encoded in transit. Content-addressed storage [14] identifies data by the bytes its codec emits, so which re-encodings preserve identity follows from the codec rather than from a declared profile. W3C PROV [17] gives a lineage vocabulary without canonical identity or verifiable freshness. Agent protocols such as MCP locate resources by URI and signal caching with server-declared TTL hints [18]; NANDA’s registry work describes agents and their data facts [19]. Our profile supplies the identity, lineage and freshness semantics those carriers lack and could reference.

VI. LIMITATIONS AND OPEN QUESTIONS

Numeric cost. The integral domain removes binary64 aliasing, including the float-spelled 9007199254740993.0, which an earlier token-only range check admitted; the cost is that producers must carry every fractional quantity as a string. We treat this as a substantial interoperability cost rather than a free choice: a schema carrying rates, prices or measurements must change to adopt the profile as specified. A future `schema_version` could admit a decimal semantic type instead, but only after specifying its lexical-to-mathematical mapping, its canonical representation, its range and precision behavior, and the equivalence relation over it; none of those is settled here. The closed payload shape likewise pushes extensions into `content` or a new `schema_version`.

Completeness. Discovering every freshness statement requires a named query, index, or monitor that no single receipt provides, and elapsed-time freshness requires service-authenticated time.

Scope of evidence. Our measurements cover the canonicalization pipeline only and come from one implementation; independent reproduction, the in-testbed evaluation, and an external pilot remain future work, and a Town-authored test is not evidence of external adoption.

Standards status. CPB is an individual Internet-Draft whose algorithm names are draft-local until RFC publication [3].

VII. CONCLUSION

A dataset identifier should survive the re-encodings agents perform, commit to the dataset’s inputs, name an owner who signed it, and take its freshness from a log rather than a clock. NFC-then-JCS identity, CPB typed references and SCITT

statements are designed to do that, and the profile is small enough to attack in an open testbed. What remains is to run it there.

DISCLOSURE

The author wrote `uor-jcs-nfc` [11], a related canonicalization context that this profile deliberately does not select; it is cited here as related work and the protocol does not depend on it. The author contributes to the UOR Foundation and founded Kessai Labs; neither organization is a dependency of the protocol, and this is personal research rather than an organizational release. One of the five closed Town contributions discussed in the requirements section, `sic_facts` (#72), was authored by the author; the requirements adopted here are taken from the maintainers’ closing reviews rather than from that contribution.

AI DISCLOSURE

We used Anthropic’s Claude to assist with drafting and editing the manuscript and with implementing the prototype and its test vectors; it materially affected all sections and the released artifact. Table I, Table II and the vector outcomes were produced by executing the released code; the author verified the correctness and originality of all content, including every reference.

REFERENCES

- [1] A. Rundgren, B. Jordan, and S. Erdtman, “JSON Canonicalization Scheme (JCS),” RFC 8785, Jun. 2020.
- [2] H. Birkholz *et al.*, “An Architecture for Trustworthy and Transparent Digital Supply Chains,” RFC 9943, Jun. 2026.
- [3] S. Mih and A. Sokolov, “Canonical Payload Binding: A Signed Statement Construction Profile,” IETF Internet-Draft draft-mih-sokolov-scitt-payload-binding-03, Sep. 2026, work in progress.
- [4] Project NANDA, “NANDA Town,” <https://github.com/projnanda/nandatown>, commit `df0b5f1fd0c2ec2f9535c786fcf83694096e4e9f`, Sep. 2026.
- [5] Maintainer closing reviews of five closed contributions to NANDA Town, 2026: pull requests #72, #50, #34, #59 and #61, <https://github.com/projnanda/nandatown>.
- [6] K. Whistler, ed., “Unicode Normalization Forms,” Unicode Standard Annex #15. <https://www.unicode.org/reports/tr15/>
- [7] T. Bray, “The I-JSON Message Format,” RFC 7493, Mar. 2015.
- [8] J. Schaad, “CBOR Object Signing and Encryption (COSE): Structures and Process,” RFC 9052, Aug. 2022.
- [9] O. Steele *et al.*, “CBOR Object Signing and Encryption (COSE) Receipts,” RFC 9942, Jun. 2026.
- [10] B. Laurie, E. Messeri, and R. Stradling, “Certificate Transparency Version 2.0,” RFC 9162, Dec. 2021.
- [11] UOR Foundation, “UOR-JCS-NFC,” v1.1 draft, 2026. <https://github.com/UOR-Foundation/uor-jcs-nfc/blob/303591128094362857791930fdc12b73875d1ff6/SPEC.md>
- [12] S. Torres-Arias *et al.*, “in-toto: Providing farm-to-table guarantees for bits and bytes,” in *Proc. USENIX Security*, 2019.
- [13] Z. Newman, J. S. Meyers, and S. Torres-Arias, “Sigstore: Software signing for everybody,” in *Proc. ACM CCS*, 2022.
- [14] J. Benet, “IPFS – Content addressed, versioned, P2P file system,” arXiv:1407.3561, 2014.
- [15] S. Mih, “An Agent Action Capsule Profile for SCITT,” IETF Internet-Draft draft-mih-scitt-agent-action-capsule-04, Aug. 2026, work in progress.
- [16] J. Gao and M. Clark, “Same Data, Different Bytes: Serialization-Invariant Integrity Checks for Agent-Published Datasets,” in *Proc. IEEE TPS*, 2026, to appear.
- [17] T. Lebo, S. Sahoo, and D. McGuinness, eds., “PROV-O: The PROV Ontology,” W3C Recommendation, Apr. 2013.
- [18] Model Context Protocol, “Specification, version 2026-07-28,” <https://modelcontextprotocol.io/specification/2026-07-28>.
- [19] R. Raskar *et al.*, “Beyond DNS: Unlocking the Internet of AI Agents via the NANDA Index and Verified AgentFacts,” arXiv:2507.14263, 2025.
- [20] M. Clark, “Canonical Dataset Lineage: Reproducibility Artifact,” 2026. <https://github.com/maurathat/canonical-dataset-lineage/tree/0a68213b2927ce160d366ddef2bfff18b094b2add>.